```
        "indices": ["vintage_cars","power_cars","rare_cars","luxury_cars"],
        "alias": "high_end_cars_alias"
      }
    }
  ]
}
```

Here, `actions` creates an alias called `high_end_cars_alias` that points to four car indexes (`vintage_cars`, `power_cars`, `rare_cars`, and `luxury_cars`). Now that we've mastered the art of creating and aliasing indexes, let's look at how to read them.

6.3 Reading indexes

So far, the indexes we've looked at are *public* indexes—those usually created by users or applications to hold data. We discuss public indexes in the next section and then look at another type of index: the hidden index.

6.3.1 Reading public indexes

We can fetch the details of an index by simply issuing a `GET` command (like `GET cars`), as we've already seen. The response provides mappings, settings, and aliases as JSON objects. The response can also return the details of multiple indexes. Say we want to return the details of three indexes: `cars1`, `cars2`, and `cars3`. The following listing shows the way forward.

Listing 6.21 Getting index configurations for multiple indexes

```
GET cars1,cars2,cars3
```

Retrieves details of three indexes

NOTE Kibana complains if the multiple-indexes URL has a space between the indexes. For example, `GET cars1, cars2` will fail because there's a space after the comma. Be sure multiple indexes are separated by just a comma.

This command returns relevant information for all three indexes, but providing a long list of comma-separated indexes does not necessarily sit well with developers. Instead, we can use wildcards if our indexes have a pattern. For example, the code in the following listing fetches all indexes starting with the letters *ca*.

Listing 6.22 Getting multiple index configurations with a wildcard

```
GET ca*
```

Returns all indexes prefixed with ca

Using the same principle with comma-separated indexes and wildcards, we can get the configurations for specific indexes. For example, the following listing gets all indexes prefixed with *mov* and *stu* (short for *movies* and *students*).